Lab Assignment 3

Title:

To execute and demonstrate *Integrity Constraints* in SQL. (**Primary Key**, **Foreign Key**, **Unique**, **Not Null**, **Check**, and **Default)**

Objective:

To understand and apply different types of **integrity constraints** — *Primary Key*, *Foreign Key*, *Unique*, *Not Null*, *Check*, *and Default* — and to ensure data consistency across multiple related tables.

Problem Statement:

Design and create a small database for an **Electronic Store Management System** with multiple related tables. Apply suitable constraints to maintain data integrity and demonstrate their behaviour using different SQL queries.

Tables to be Created:

1. `Customer` — Customer details
2. `Items` — Product details
3. `Orders` — Order header table
4. `Order_Details` — Order line Itemss (transaction details)

1. Create Table: Customer

```
CREATE TABLE Customer (
    CustId INT PRIMARY KEY,
    CustName VARCHAR(40) NOT NULL,
    City VARCHAR(30) DEFAULT 'Aurangabad',
    ContactNo CHAR(10) UNIQUE CHECK (LEN(ContactNo) = 10)
);
```

Explanation:

- `CustId` → Primary Key
- `CustName` → Cannot be null
- `City` → Default value 'Aurangabad'
- `ContactNo` → Must be unique and exactly 10 digits

2. Create Table: Items

```
CREATE TABLE Items (
    ItemId INT PRIMARY KEY,
    ItemsName VARCHAR(30) UNIQUE NOT NULL,
    Price DECIMAL(10,2) CHECK (Price > 0),
    Category VARCHAR(25) DEFAULT 'Electronics'
);
```

Explanation:

- `ItemsName` → Unique and not null
- `Price` → Must be positive
- `Category` → Default category if not specified

3. Create Table: Orders

```
CREATE TABLE Orders (
    OrderId INT PRIMARY KEY,
    CustId INT NOT NULL,
    OrderDate DATE,
    FOREIGN KEY (CustId) REFERENCES Customer(CustId)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
```

Explanation:

- Foreign Key maintains referential integrity with `Customer`.
- **Cascade options** ensure related rows update/delete automatically.

4. Create Table: Order_Details

```
CREATE TABLE Order_Details (
    OrderId INT,
    ItemId INT,
    Quantity INT CHECK (Quantity > 0),
    Discount DECIMAL(5,2) DEFAULT 0,
    PRIMARY KEY (OrderId, ItemId),
    FOREIGN KEY (OrderId) REFERENCES Orders(OrderId),
    FOREIGN KEY (ItemId) REFERENCES Items(ItemId)
);
```

Explanation:

- Composite Primary Key → `(OrderId, ItemId)`
- Foreign Keys link with both `Orders` and `Items` tables.
- Ensures valid quantity and optional discount field.

5. Insert Valid Records

```
INSERT INTO Customer VALUES (1, 'Rahul Sharma', 'Aurangabad',
'9876543210');
INSERT INTO Customer VALUES (2, 'Sneha Patil', 'Pune', '9823456789');
INSERT INTO Customer VALUES (3, 'Amit Deshmukh', DEFAULT, '9991122334');

INSERT INTO Items VALUES (101, 'Keyboard', 850.00, 'Computer');
INSERT INTO Items VALUES (102, 'Mouse', 450.00, 'Computer');
INSERT INTO Items VALUES (103, 'Monitor', 12500.00, 'Display');

INSERT INTO Orders VALUES (1001, 1, '2025-10-10');
INSERT INTO Orders VALUES (1002, 2, '2025-10-11');

INSERT INTO Order_Details VALUES (1001, 101, 2, 5.0);
INSERT INTO Order_Details VALUES (1001, 102, 1, 0);
INSERT INTO Order_Details VALUES (1002, 103, 1, 10.0);
```

6. Demonstrate Constraint Violations

a) **Check Constraint Violation:**

```
INSERT INTO Items VALUES (104, 'USB Cable', -100.00, 'Accessory');
-- Error: Check constraint violated (Price > 0)
```

b) **Unique Constraint Violation:**

```
INSERT INTO Customer VALUES (4, 'Rohit Jadhav', 'Pune', '9876543210');
-- Error: Duplicate ContactNo not allowed.
```

c) **Foreign Key Violation:**

```
INSERT INTO Order_Details VALUES (1005, 105, 2, 0);
-- Error: No matching OrderId or ItemId exists.
```

7. Queries Using Constraints

a) **Display all orders with customer and Items details**

```
SELECT o.OrderId, c.CustName, i.ItemsName, od.Quantity, i.Price,
       (i.Price * od.Quantity) - od.Discount AS TotalAmount
FROM Customer c
JOIN Orders o ON c.CustId = o.CustId
JOIN Order_Details od ON o.OrderId = od.OrderId
JOIN Items i ON od.ItemId = i.ItemId;
```

b) **Find total purchase amount by each customer**

```
SELECT c.CustName, SUM((i.Price * od.Quantity) - od.Discount) AS
TotalPurchase
FROM Customer c
JOIN Orders o ON c.CustId = o.CustId
JOIN Order_Details od ON o.OrderId = od.OrderId
JOIN Items i ON od.ItemId = i.ItemId
GROUP BY c.CustName
HAVING SUM((i.Price * od.Quantity) - od.Discount) > 10000;
```

c) **Display all customers who placed orders in October 2025**

```
SELECT DISTINCT c.CustName, c.City
FROM Customer c
JOIN Orders o ON c.CustId = o.CustId
WHERE MONTH(o.OrderDate) = 10
AND YEAR(o.OrderDate) = 2025;
```

Result:

All integrity constraints — *Primary Key, Foreign Key, Unique, Not Null, Check,* and *Default* — were successfully demonstrated on multiple related tables. The relationships maintained data consistency and supported complex multi-table queries effectively.